
Policy Overflow: Distance-to-Manifold Corrections for Conditional Flow-Matching Policies

Edward W. Staley

Johns Hopkins Applied Physics Laboratory
Laurel, MD 20723
edward.staley@jhuapl.edu

Abstract

Flow matching is a common recipe for generative policies trained with imitation learning, since it can capture multiple modes of demonstration. However, flow matching is still capable of producing erroneous points which are caught in stagnant regions of flow, resulting in the generation of out-of-distribution actions that may degrade policy performance. We present Policy Overflow, a method that learns a separate distance-to-manifold model to correct underestimated velocity during generation. We demonstrate this correction in two-dimensional distributions which are easy to visualize, showing the undesirable behavior of standard flow matching and the impact of our correction. We then show that this correction remains useful in higher-dimensional and state-conditioned policy learning problems, and that learning a separate distance model is a better use of model parameters than simply increasing the parameter count of the flow matching policy. We explore a range of diffusion transformer (DiT) model scales and design choices on PushT, and additionally verify the best approaches using select tasks in Robomimic.

1 Introduction and Background

Flow matching [1] has become a popular choice for generative modeling due to its expressiveness and simplicity. In this technique, a time-varying velocity field is learned which transports data points from a known distribution to a target distribution. For a datapoint x^t at a time $t \in [0, 1]$ in the generative process, a model $v_\theta(x^t, t)$ learns to output a velocity $v = dx/dt$, and Euler integration can be used to step the generation forward: $x^{t+1} = x^t + v * dt$ and $t = t + dt$.

Following Diffusion Policy [2], conditional flow matching has also become an integral technique for generative imitation learning policies [3], in which the model is conditioned on the current state and generates actions. Two critical differences to unconditional generation are that (1) while the distribution of actions is often multimodal, a dataset only pairs one action with a given state, and (2) during inference time the goal is to generate actions once for an observed state, rather than to provide access to a distribution that can be explored or re-sampled.

In this preliminary work, we explore the training of secondary models that learn a distance-to-manifold field that corrects and improves such policies. Related prior work includes Distance Marching [4], which learns a similar distance field for image distributions and uses this field directly for generation. Cai et al. show that the ideal velocity field is proportional to the negative gradient of a squared distance function, and introduce a classifier-free guidance (CFG [5]) technique that generates images by projecting points onto a manifold [6]. Prior work in offline reinforcement learning (RL) considers learned distance-to-data functions as a means to improve policy learning: DOGE [7] uses this as a constraint when learning a Q-value function, and SGP [8] uses distance-to-data to guide planning in model-based offline RL.

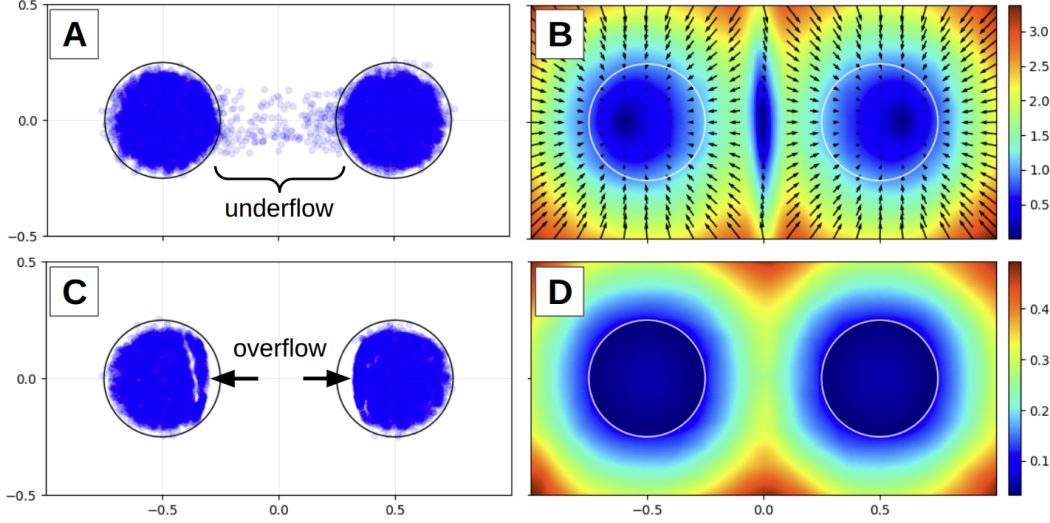


Figure 1: Illustration of the underflow problem and our solution in a constructed bimodal example. **A:** A standard flow-matching model will sometimes generate points that do not reach the data manifold. **B:** The corresponding learned velocity field toward the end of generation, showing velocity with black arrows and magnitude error as a heatmap. We clearly see velocity underestimated around the y-axis. **C:** Our proposed overflow correction, which pushes points to the manifold and sometimes beyond. **D:** The learned distance field which is used to correct the flow for this example.

2 Methods and Illustrative Examples

We propose learning a distance-to-manifold function to correct for stray point generations in flow matching, which can result from underestimated velocity. This phenomenon is described in The Velocity Deficit [9], which proposes to correct it by either learning amplified magnitudes or amplifying them during generation according to a schedule.

Our method takes a more direct approach: predict the distance to the data manifold, and measure if a given velocity is underestimated. If so, use the predicted distance remaining to amplify it such that the point would reach the data manifold. We call this method policy *overflow*, since we increase the amount of flow during generation. In some regions this may under-sample the borders of the target distribution (see Figure 1 C), but helps avoid out-of-distribution samples that do not reach the target distribution in the first place ("underflow"). This is important for flow policies, in which a single action that does not reach the data manifold may push a policy into an unrecoverable state.

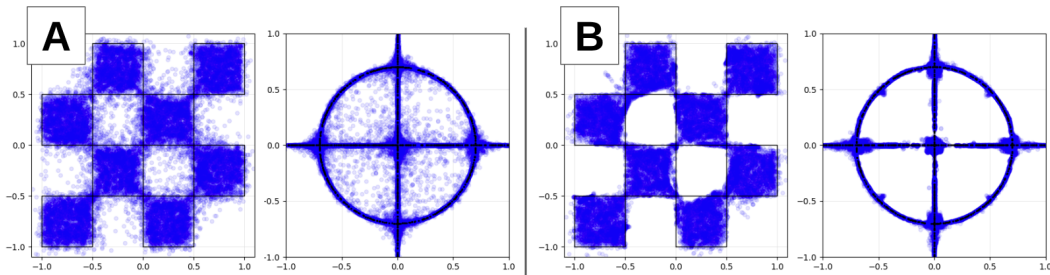


Figure 2: Our overflow correction on additional 2D examples, showing a qualitative decrease in erroneous points. **A:** Standard flow matching on a checkerboard dataset and a crosshair dataset, each showing 20k generated samples in blue. **B:** Samples from the same models when a learned distance field is used to correct generation.

Figure 1 illustrates the velocity underestimation problem and our distance correction in a constructed dataset with two modes. We provide additional two-dimensional examples in figure 2, which more fully shows the qualitative impact of our method. Given many generated samples, the standard flow-matching model produces a fuzzy distribution with points that have not reached the data manifold, while our correction pushes such points closer to the target distribution.

Quantitatively, we train a secondary model $D_\phi(x^t|c)$ with a similar architecture to the flow matching model, which learns distance-to-manifold components by training on noised points within a local region, as well as on the boundary condition that target datapoints have distance zero:

$$\mathcal{L}_D = \mathbb{E} \left[\|D_\phi(x^t + \epsilon|c) - |\epsilon|\|_2^2 \right] + \mathbb{E} \left[\|D_\phi(x^t|c)\|_2^2 \right]$$

where x is from the target distribution, ϵ is some sample from a downscaled noise distribution (we use $0.1 * \mathcal{N}(0, 1)$), and c is some state-conditional. c is not present for our static two-dimensional examples but is a placeholder for state information in the policy case. During inference, we perform a reachability check along each velocity component: the predicted distance component replaces the velocity magnitude component if the manifold would not be reached under a one-step integration step. We apply the following correction to each component:

$$\|v_{corrected}\|_2 = \max(\|v_\theta(x^t|c, t)\|_2, (D_\phi(x^t|c) - \lambda) \frac{1}{1-t})$$

where $v_\theta(x^t|c, t)$ is the predicted flow matching velocity component under condition c at generative timestep t . λ is an optional small constant to adjust the distance correction threshold; we use 0.05 in the two-dimensional experiments and discard it in later policy experiments. $1/(1-t)$ is necessary to account for the velocity being scaled by dt when generation steps forward. Training details for two-dimensional experiments are provided in the appendix A.2.1.

3 Flow Matching Policy Experiments

3.1 PushT Scaling Study

We begin exploration of our method applied to policies by training a range of model sizes on the common PushT problem [10, 2, 11]. We are interested in two questions: (1) is the addition of a separate distance model worth the additional parameter count, and (2) how should we allocate parameters between the two models?

To investigate the first question, we run many trials using a flow matching model and a distance model of the same size, and compare performance between the uncorrected flow matching model and the policy overflow correction. We use DiT backbones [12] with sizes of approximately 1, 3, 5, 10, 20, 40, and 80 million (M) parameters, such that the flow matching models span 1 to 80M DiT parameters and the total Policy Overflow parameters span 2 to 160M. For every size in question we run five trials, selecting the best performing checkpoint of the flow matching and paired flow matching + distance curves. The resulting performance vs parameter are given in Figure 3 left.

To better understand question (2), we also try pairing flow matching models with a range of distance model sizes. We test a 10M parameter flow matching model with distance models of size 3, 5, and 10M, and a 50M parameter flow matching model with distance models of size 3, 5, 10, 25, and 50M parameters. These results are given in Figure 3 right. As a secondary experiment, we also use these sizes to explore the benefit of correcting each action component separately vs correcting all components together. These results are shown in orange in Figure 3 right. Details of our model training, including architecture and hyperparameters, can be found in the appendix A.2.2.

3.2 PushT and Robomimic Tasks

Our PushT scaling results indicated that a smaller distance model could still yield a useful correction, and that this was potentially a better use of parameters than increasing the size of the flow matching model. To confirm this, we applied our method to three additional tasks from Robomimic [13] that were investigated in the original diffusion policy work: Square, Transport, and Tool Hang. We use the multi-human (MH) datasets where possible, but only the proficient-human (PH) dataset was available for Tool Hang. We omit the Can and Lift tasks as they are considerably easier and may not serve to highlight any differences in performance.

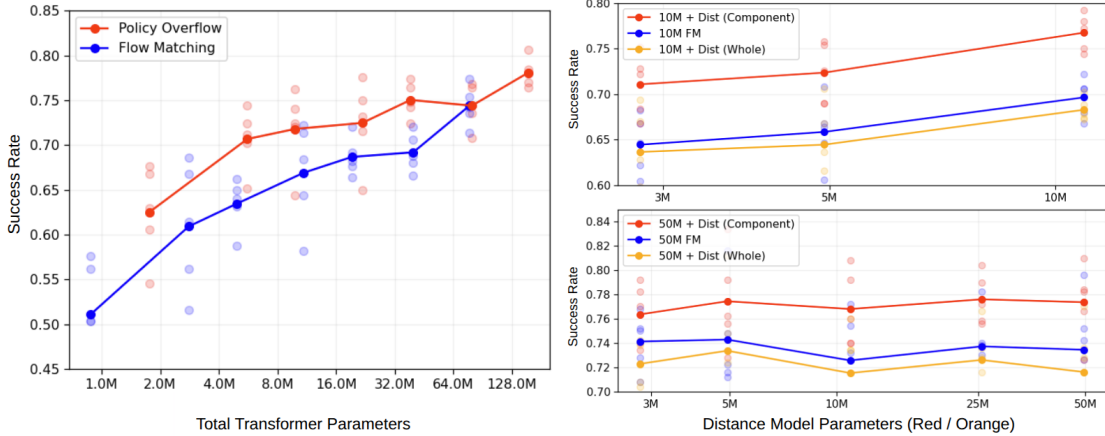


Figure 3: Scaling experimental results on PushT. **Left:** Comparison of flow matching (blue) and Policy Overflow (red), when performance is plotted against parameters. **Right:** Experiments holding the flow matching model fixed at 10M parameters (top) and 50M (bottom), while varying the size of the distance model. We also test whole-vector correction as an alternative to component-wise corrections (orange).

The results of these experiments are given in Table 1. As before, we run five trials per result and take an average. Details of model training are available in the appendix A.2.3.

Approach	PushT	Square (MH)	Transport (MH)	Tool Hang (PH)
<i>FM 20M</i>	0.653 / 0.640	0.734 / 0.703	0.440 / 0.415	0.894 / 0.880
<i>FM 25M</i>	0.712 / 0.697	0.722 / 0.694	0.434 / 0.405	0.902 / 0.885
<i>PO 20+5M</i>	0.739 / 0.725	0.779 / 0.748	0.482 / 0.457	0.935 / 0.921
<i>Task Notes</i>	Vision+State	State Only	State Only	State Only

Table 1: Results on PushT and robomimic Square, Transport, and Tool Hang tasks, using a 20M parameter flow matching model with a 5M parameter distance model applying our policy overflow correction ("PO 20+5M"). For comparison, we include results using the 20M flow matching model without correction ("FM 20M") and a flow matching model with similar total parameters ("FM 25M"). Results include the best checkpoint during training and an average of the top five checkpoints, given as top-1 / top-5. All entries are averages over five independent runs.

4 Results and Discussion

We find broadly that Policy Overflow provides a meaningful performance boost in all experiments, and this seems to hold across a range of model sizes. In section 3.1, we see improved performance vs parameter count for the trials that utilize the distance model correction compared to the baseline flow matching, with the only point of overlap at 80M parameters. As this is towards the high end of our parameter range, further experimentation is needed to explore this trend into the hundreds of millions of parameters. In section 3.2, we see a boost of several percentage points in all four tested tasks which use the 20M flow matching + 5M distance model formula, even when compared to independent runs of a 25M parameter flow matching model. Additionally, this result is not a matter of outliers in evaluation; we see the same result when aggregating the top five checkpoints from a given trial.

While further exploration is needed at higher parameter counts and higher dimension tasks, these preliminary results suggest that vanilla flow matching policies have a tendency to learn suboptimal flow fields that can benefit from direct intervention. A natural future question is whether our correction could be folded back into a single model, either via distillation or a change in the original flow matching objective.

Acknowledgments and Disclosure of Funding

The authors gratefully acknowledge internal financial support from the Johns Hopkins University Applied Physics Laboratory's Independent Research and Development (IRAD) Program for funding portions of this work.

References

- [1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” 2023.
- [2] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” 2024.
- [3] X. Hu, B. Liu, X. Liu, and Q. Liu, “Adaflow: Imitation learning with variance-adaptive flow-based policies,” 2024.
- [4] Z. Wang, I. Mehta, H. Lu, C.-E. Sun, G. Yan, T.-W. Weng, and T.-M. Li, “Distance marching for generative modeling,” 2026.
- [5] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” 2022.
- [6] J.-F. Cai, H. Liu, Z. Su, and C. Wang, “Improving classifier-free guidance of flow matching via manifold projection,” 2026.
- [7] J. Li, X. Zhan, H. Xu, X. Zhu, J. Liu, and Y.-Q. Zhang, “When data geometry meets deep function: Generalizing offline reinforcement learning,” 2023.
- [8] H. J. T. Suh, G. Chou, H. Dai, L. Yang, A. Gupta, and R. Tedrake, “Fighting uncertainty with gradients: Offline reinforcement learning via diffusion score matching,” 2023.
- [9] L. Li, Z.-W. Hong, S. Zhang, B. Lin, J. Li, Y. Tang, and J. Liang, “The velocity deficit: Initial energy injection for flow matching,” 2026.
- [10] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, “Implicit behavioral cloning,” 2021.
- [11] R. Cadene, S. Aliberts, F. Capuano, M. Aractingi, A. Zouitine, P. Kooijmans, J. Choghari, M. Russi, C. Pascal, S. Palma, M. Shukor, J. Moss, A. Soare, D. Aubakirova, Q. Lhoest, Q. Gallouédec, and T. Wolf, “Lerobot: An open-source library for end-to-end robot learning,” 2026.
- [12] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” 2023.
- [13] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2021.

A Appendices

A.1 Formal Description of Models

We can generically define a (conditional) flow matching model as $v_\theta(x^t|c, t)$, where x^t is some sample being generated by the model at timestep $t \in [0, 1]$, and c is some conditioning variable in the case that the model is conditional. In our two-dimensional experiments, we do not have c , and our model is described succinctly as $v_\theta(x^t, t)$, using v to denote that the output may be treated as a velocity.

In the case of a policy, we often use different variables to otherwise define the same architecture: $v_\theta(x^t|c, t)$ becomes $\pi_\theta(A^t|O, t)$ with direct correspondence between the variables in order. Given that the policy is trained and deployed on data with environmental timestep k , we define a flow matching policy network $\pi_\theta(A_k^t|O_k, t)$ which generates a short horizon of future actions $A_k^{t=1.0} = [a_k, a_{k+1}, a_{k+2}, \dots, a_{k+H-1}]$ over some horizon H . We use $H = 16$ in all our experiments, and the dimensionality of an individual a_k is provided by the task. The policy is conditioned on the current observation information from the environment which we aggregate over two timesteps: $O_k = [I_{k-1}, I_k, q_{k-1}, q_k]$, where I is an image observation (only present for PushT), and q is a low level state vector, such as proprioceptive information.

Our distance models use an identical architecture as the flow matching generators but do not condition on t . For the two-dimensional case we define $D_\phi(x^t)$ and for the policy case $D_\phi(A_k^t|O_k)$.

A.2 Architectures and Training Details

A.2.1 Two Dimensional Examples

Our two-dimensional examples are trained with lightweight MLP models with two hidden layers of size 256. We learn a 16-dimensional time embedding for generation, and simply input this alongside the two-dimensional datapoint. We sample from a true distribution rather than constructing a fixed dataset, and train for 30k updates using a batch size of 256, the Adam optimizer, and a learning rate of $1e-3$. When used, the distance model is simply trained alongside the flow matching model using a similar MLP (without time conditioning) and using the same target datapoints.

A.2.2 Flow Policy Architecture

For all of the experiments in Section 3 we use a DiT architecture that accepts two prior states, two prior images (for PushT), and processes 16 actions. The PushT images are fed through a ResNet extractor of approximately 1.5 million parameters that we re-train with all PushT policies. The architectures for the flow matching policy and the distance models both follow this template: the flow matching models output a velocity for each action and are conditioned on the generation step t , while the distance models output distance components for each action and do not have this condition. We use a schedule of 16 generation steps and replan every 4 actions, discarding the remain 12 from the action chunk. Our model architectures are also provided as a diagram in Figure 4.

A.2.3 Flow Policy Training Parameters

Our flow policy experiments train for different lengths depending on the task, in an attempt to highlight when the flow matching policy peaks in performance. The PushT trials all train for 400 epochs and the distance model is trained in parallel on the same batches of data. The Square and Tool Hang tasks train for 50 epochs, and the distance model, when used, is warm-started for 50 epochs before training alongside the policy for 50 epochs. Similarly, Transport trains for 150 epochs and the distance model warm-starts for an additional 50 epochs. These epoch counts were determined by initially using the same 400-epoch recipe from PushT and finding that performance plateaued and then degraded very early, sometimes before the distance model had converged (hence the warm starting). For all policies we use the Adam optimizer with a learning rate of $1e-4$ and a batch size of 64.

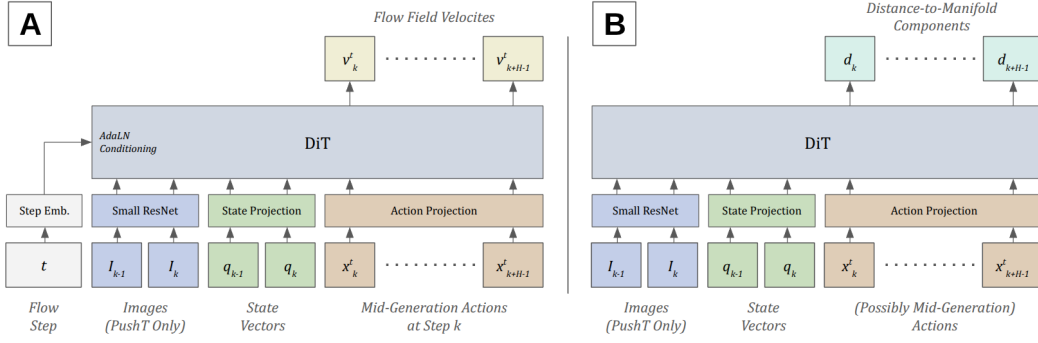


Figure 4: Model architecture used for all flow matching policy experiments. A: The DiT used for policies, which conditions on the generation step t . The same transformer parameters are used for states and actions. B: The distance model architecture, which is identical other than not conditioning on t .